

Python Backend Development Basics

Chapter 1: APIs and Services

A backend service receives requests, performs business logic, accesses data or other services, and returns responses. REST APIs commonly use HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

Good backend design separates concerns. Configuration, business logic, data access, external service clients, and request handling should not all be placed in one large file.

Key Points

- GET is commonly used to retrieve resources.
- POST is commonly used to create or trigger an operation.
- Separation of concerns improves maintainability.

Chapter 2: Authentication and Security

Authentication verifies who a user is, while authorization determines what an authenticated user is allowed to do. APIs should validate credentials and enforce authorization before performing protected operations.

Secrets such as API keys should not be hard-coded in source code. Environment variables or a secrets manager can be used to provide sensitive configuration at runtime.

Input validation, secure error handling, logging, rate limiting, and HTTPS are common security considerations for backend applications.

Key Points

- Authentication and authorization are different concepts.
- Never commit API keys to source control.
- Validate external input before processing it.

Chapter 3: Testing and Observability

Testing verifies that application behavior matches its requirements. Unit tests focus on small components, while integration tests verify interactions between components such as a database and an API.

Logging provides information about application behavior and failures. Metrics can measure request latency, error rates, and resource usage. Tracing can help follow a request across multiple services.

Key Points

- Unit tests isolate small pieces of logic.
- Integration tests verify component interactions.
- Logs, metrics, and traces support debugging and monitoring.